

Exercice : Translation puis rotation

Consigne

Reprendre l'exercice précédent et écrire une seconde fonction qui applique la translation d'abord et la rotation ensuite. Afficher les deux résultats pour le même point. Puis trouver une pose et un point pour lesquels les deux coïncident, et dire pourquoi.

Rappel

La fonction correcte applique la rotation puis la translation :

$$p_{\text{espace}} = R \cdot p + t$$

La seconde fonction, incorrecte pour une pose, applique la translation puis la rotation :

$$p' = R \cdot (p + t)$$

Les deux fonctions

```
// Fonction correcte : rotation puis translation.
Vec3 AppliquerRotationPuisTranslation(const Pose& pose, const Vec3& p)
{
    Vec3 tourne = Rotation(pose.orientation, p);
    return {
        tourne.x + pose.position.x,
        tourne.y + pose.position.y,
        tourne.z + pose.position.z
    };
}

// Fonction incorrecte : translation puis rotation.
Vec3 AppliquerTranslationPuisRotation(const Pose& pose, const Vec3& p)
{
    Vec3 translate = {
        p.x + pose.position.x,
        p.y + pose.position.y,
        p.z + pose.position.z
    };
    return Rotation(pose.orientation, translate);
}
```

Programme complet

```
#include <array>
#include <cstdio>

struct Vec3 { float x, y, z; };
struct Quat { float w, x, y, z; };
struct Pose { Vec3 position; Quat orientation; };

Vec3 Rotation(const Quat& q, const Vec3& v)
{
    Vec3 u = { q.x, q.y, q.z };
    Vec3 uv = {
        u.y * v.z - u.z * v.y,
        u.z * v.x - u.x * v.z,
        u.x * v.y - u.y * v.x
    };
}
```

```

        u.x * v.y - u.y * v.x
    };
    Vec3 t = {
        u.y * uv.z - u.z * uv.y + q.w * v.x,
        u.z * uv.x - u.x * uv.z + q.w * v.y,
        u.x * uv.y - u.y * uv.x + q.w * v.z
    };
    return {
        v.x + 2.0f * t.x,
        v.y + 2.0f * t.y,
        v.z + 2.0f * t.z
    };
}

Vec3 RotationPuisTranslation(const Pose& pose, const Vec3& p)
{
    Vec3 tourne = Rotation(pose.orientation, p);
    return {
        tourne.x + pose.position.x,
        tourne.y + pose.position.y,
        tourne.z + pose.position.z
    };
}

Vec3 TranslationPuisRotation(const Pose& pose, const Vec3& p)
{
    Vec3 translate = {
        p.x + pose.position.x,
        p.y + pose.position.y,
        p.z + pose.position.z
    };
    return Rotation(pose.orientation, translate);
}

int main()
{
    Pose pose;
    Vec3 point;

    std::scanf("%f %f %f", &pose.position.x,
                  &pose.position.y,
                  &pose.position.z);
    std::scanf("%f %f %f %f", &pose.orientation.w,
                  &pose.orientation.x,
                  &pose.orientation.y,
                  &pose.orientation.z);
    std::scanf("%f %f %f", &point.x, &point.y, &point.z);

    Vec3 r1 = RotationPuisTranslation(pose, point);
    Vec3 r2 = TranslationPuisRotation(pose, point);

    std::printf("Rotation puis translation : %.4f %.4f %.4f\n",
                r1.x, r1.y, r1.z);
    std::printf("Translation puis rotation : %.4f %.4f %.4f\n",
                r2.x, r2.y, r2.z);

    return 0;
}

```

Exemple d'exécution

Pour l'entrée :

```
1.0 2.0 3.0
0.7071 0.0 0.7071 0.0
1.0 0.0 0.0
```

Le quaternion est une rotation de 90 autour de l'axe y . Les deux résultats diffèrent :

```
Rotation puis translation : 1.0000 2.0000 2.0000
Translation puis rotation : 4.0000 2.0000 -1.0000
```

Cas où les deux coïncident

Les deux fonctions coïncident lorsque :

$$R \cdot p + t = R \cdot (p + t)$$

En développant :

$$R \cdot p + t = R \cdot p + R \cdot t$$

Donc :

$$t = R \cdot t$$

C'est-à-dire que la translation t doit être **invariante par la rotation R** . Deux cas principaux :

1. **Translation nulle** : $t = (0, 0, 0)$. Alors les deux fonctions se réduisent à la rotation seule.
2. **Translation colinéaire à l'axe de rotation** : si t est parallèle à l'axe du quaternion, alors $R \cdot t = t$.
Par exemple, une rotation autour de l'axe y avec une translation selon y .

Exemple concret. Pose : position $(0, 5, 0)$, orientation = rotation de 90 autour de l'axe y . Point : $(1, 0, 0)$.

- Rotation puis translation : on tourne $(1, 0, 0)$ autour de y de 90, ce qui donne $(0, 0, -1)$, puis on ajoute $(0, 5, 0)$: résultat $(0, 5, -1)$.
- Translation puis rotation : on ajoute $(0, 5, 0)$ à $(1, 0, 0)$, ce qui donne $(1, 5, 0)$, puis on tourne autour de y de 90 : $(1, 5, 0)$ devient $(0, 5, -1)$.

Les deux résultats sont identiques : $(0, 5, -1)$.

Pourquoi

La translation $(0, 5, 0)$ est parallèle à l'axe de rotation y . Une rotation autour de y ne modifie pas la composante y d'un vecteur, et ne mélange pas y avec x ou z . Donc $R \cdot t = t$, et les deux fonctions coïncident.

À retenir

L'ordre correct est **rotation puis translation**. L'ordre inverse fait tourner l'objet autour de l'origine du monde au lieu de le faire tourner sur lui-même. Le point autour duquel la rotation s'effectue s'appelle le **pivot**. Les deux ordres ne coïncident que dans des cas particuliers : translation nulle, ou translation parallèle à l'axe de rotation.